CEDT Lab 10 - Tasks, Lock and Queue

จุดประสงค์การเรียนรู้

1. เข้าใจการทำงานคร่าวๆของ RTOS
2. สามารถใช้ Task เพื่อทำงานหลายงานพร้อมๆกัน
3. สามารถใช้ Lock เพื่อแบ่งการใช้งานของ Resource
4. สามารถใช้ Queue เพื่อส่งข้อมูลระหว่าง Task

อุปกรณ์ที่ใช้ในการทดลอง

1. ตัวต้านทานค่าต่างๆ (220 Ohm , 1k Ohm, 10kOhm)
2. LEDs (Diode)
3. สายไฟ
4. ESP32
5. Switch หรือ ปุ่ม

Real-Time Operating System

ระบบปฏิบัติการแบบเวลาจริง หรือ **ระบบปฏิบัติการแบบทันที** (Real-time operating system: RTOS) คือระบบปฏิบัติการที่เวลาเป็นปัจจัยสำคัญสำหรับประสิทธิภาพของระบบ นั่นคือมีชุดคำสั่งหรือโปรเซสบางอย่าง ที่จำเป็นต้องทำ ณ เวลาที่กำหนด หรือทำให้เสร็จในเวลาที่กำหนด ถ้าทำไม่ได้อาจสร้างความเสียหายหรือเกิดค่าความเสียหาย (cost) กับระบบ โดยทั่วไปสำหรับระบบปฏิบัติการแบบเวลาจริง การทำงานไม่ทันในเวลาที่กำหนด มีค่าเทียบเคียงได้กับ การไม่ได้ทำงานนั้นเลย ระบบปฏิบัติการแบบเวลาจริงมักถูกออกแบบมาเพื่อแอปพลิเคชันแบบฝังตัวหรือแอปพลิเคชันที่ติดตั้งมากับระบบตั้งแต่แรกเพื่อทำงานเฉพาะด้าน เช่น ระบบปฏิบัติการของเครื่องเล่นซีดี ทันทีที่ระบบปฏิบัติการอ่านข้อมูลจากแผ่นซีดีแล้ว จะต้องประมวลผลข้อมูล และส่งสัญญาณเสียงออกไปให้ผู้ฟังทันที ถ้าระบบปฏิบัติการทำงานไม่ทันก็จะทำให้เพลงฟังไม่รู้เรื่อง (Wiki)

ESP32 ดัดแปลง FreeRTOS v10.5.1 ซึ่งเป็น RTOS ที่มีความนิยมสูงแต่สร้างมาสำหรับการทำงานของ CPU เดียว ESP-IDF FreeRTOS จึงมีคำสั่งที่และการเรียกที่แตกต่างจาก FreeRTOS เล็กน้อย

ใน Lab นี้จะให้ทดลองใช้ RTOS สำหรับการทำงานสามอย่าง Task, Mutex (i.e. Lock) และ Queue

การทดลอง

1. **Tasks**

Tasks หรือ Threads เป็นการที่ OS จัดแบ่งเวลาให้หลายๆ Tasks สามารถทำงานพร้อมกันได้ เนื่องจาก ESP32 มี Wifi และ Bluetooth Stacks ซึ่งต้องมีการทำงานตลอดเวลาจึงมี Tasks ที่ทำงานเป็น Background (ทำงานไปเรื่อยๆตลอด) อยู่แล้ว

ตัวอย่าง Code ที่สามารถ สร้างสอง Task ที่ทำงานพร้อมกันได้

```
#include <Arduino.h>

TaskHandle_t Task1;
TaskHandle_t Task2;

// LED pins
const int led1 = 2;
```

```
const int led2 = 4;


//Task1code: blinks an LED every 1000 ms
void Task1code( void * pvParameters ){
 Serial.print("Task1 running on core ");
 Serial.println(xPortGetCoreID());

 for(;;){
   digitalWrite(led1, HIGH);
   delay(1000);
   digitalWrite(led1, LOW);
   delay(1000);
 }
}

//Task2code: blinks an LED every 700 ms
void Task2code( void * pvParameters ){
 Serial.print("Task2 running on core ");
 Serial.println(xPortGetCoreID());

 for(;;){
   digitalWrite(led2, HIGH);
   delay(700);
   digitalWrite(led2, LOW);
   delay(700);
 }
}


void setup()
{
  Serial.begin(115200);

 pinMode(led1, OUTPUT);
 pinMode(led2, OUTPUT);

 //create a task that will be executed in the Task1code() function, with
priority 1 and executed on core 0
```

```
 xTaskCreate(
                  Task1code,   /* Task function. */
                  "Task1",     /* name of task. */
                  10000,       /* Stack size of task */
                  NULL,        /* parameter of the task */
                  1,           /* priority of the task */
                  &Task1      /* Task handle to keep track of created
task */
                  );
 delay(500);

 //create a task that will be executed in the Task2code() function, with
priority 1 and executed on core 1
 xTaskCreatePinnedToCore(
                  Task2code,   /* Task function. */
                  "Task2",     /* name of task. */
                  10000,       /* Stack size of task */
                  NULL,        /* parameter of the task */
                  1,           /* priority of the task */
                  &Task2,      /* Task handle to keep track of created
task */
                  1);          /* pin task to core 1 */
   delay(500);
}
void loop()
{
}
```

ESP32 สองคำสั่งที่ใช้ในการสร้าง Task สองอันคือ xTaskCreate และ xTaskCreatePinnedToCore โดยในคำสั่ง xTaskCreate ตัว Task จะทำงานจาก Core ไหนก็ได้ (Core 0, 1) ส่วน xTaskCreatePinnedToCore สามารถระบุได้ว่าให้ทำงานจาก Core ไหน

ส่งใน MCV

- แก้โปรแกรมข้างต้นให้ส่งเพิ่มเป็น 3 tasks, โดยใน Task ที่สามให้ทำการรับข้อมูลจาก Serial Terminal แล้ว Echo กลับมา ส่ง Code ที่ทำงานถูกต้องใน MCV

2. Semaphore (Mutex/Lock)

หนึ่งในหน้าที่ของ OS คือช่วยจัดการให้ Resource ถูกใช้อย่างถูกต้อง ตัวอย่างเช่น ถ้ามีโปรแกรมใช้ เครื่องพิมพ์อยู่ โปรแกรมอื่นๆ ไม่ควรจะสามารถใช้เครื่องพิมพ์ได้จนกระทั่งโปรแกรมแรกใช้งานเสร็จสิ้น ลองพิจรณา Code ต่อไปนี้

```
#include <Arduino.h>

TaskHandle_t Task1;
TaskHandle_t Task2;

//Task1code: blinks an LED every 1000 ms
void Task1code( void * pvParameters ){

 for(;;){
   Serial.print("{");
   Serial.print("This is task1");
   Serial.println("}");
 }
}

//Task2code: blinks an LED every 700 ms
void Task2code( void * pvParameters ){
 for(;;){
   Serial.print("(");
   Serial.print("This is task2");
   Serial.println(")");
 }
}

void setup()
{
  Serial.begin(115200);

 //create a task that will be executed in the Task1code() function, with
priority 1 and executed on core 0
 xTaskCreate(
                   Task1code,   /* Task function. */
                   "Task1",     /* name of task. */
                   10000,       /* Stack size of task */
                   NULL,        /* parameter of the task */
                   1,           /* priority of the task */
                   &Task1      /* Task handle to keep track of created
task */
```

```
                    );
  //create a task that will be executed in the Task2code() function,
with priority 1 and executed on core 1
 xTaskCreate(
                    Task2code,    /* Task function. */
                    "Task2",      /* name of task. */
                    10000,        /* Stack size of task */
                    NULL,         /* parameter of the task */
                    1,            /* priority of the task */
                    &Task2       /* Task handle to keep track of created
task */
                    );           /* pin task to core 1 */

}
void loop()
{
}
```

สิ่งที่คาดหวังว่าจะเป็น คือ การเรียงลำดับของ Output ถูกต้อง จะพบว่าในบางครั้งมีการทำงานที่ผิดลำดับ

จงใช้ Semaphore (ดูตัวอย่างใน xSemaphoreTake

https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32/api-reference/system/freertos.html#semaphore-api)

ส่งใน MCV

- ส่ง Code และ Record การทำงาน จาก serial terminal

3. Queue

ใน Lab นี้ให้สร้าง Task สอง Task

- Task 1 รับข้อมูลจาก Serial และใส่ Queue
- Task 2 เขาข้อมูลจาก Queue แล้ว พิมพ์ใน Serial

ให้ใช้ Queue จาก xQueueGenericSend, xQueueReceive

(https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32/api-reference/system/freertos.html#queue-api)

ส่งใน MCV

- ส่ง Code และ Record การทำงาน จาก serial termina